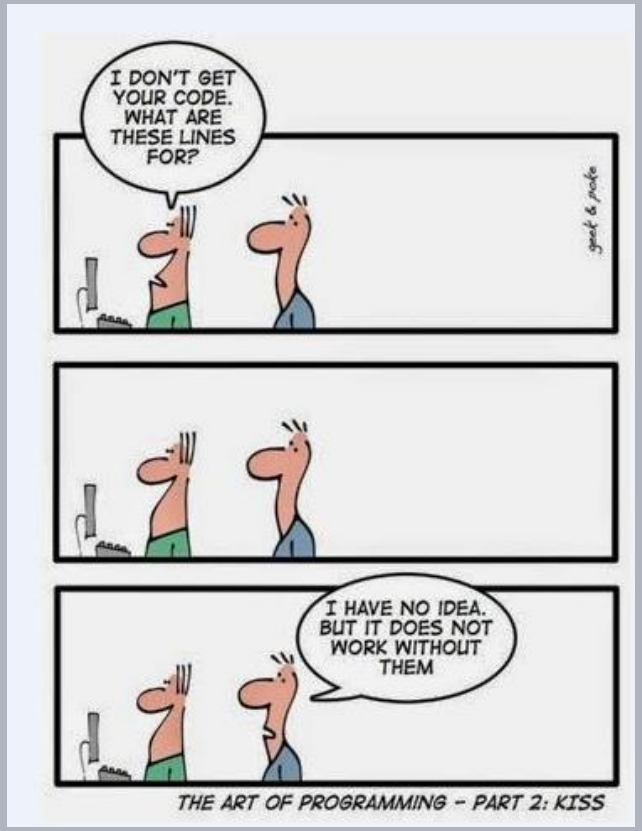
Advanced Software Design Techniques

UML, Object Oriented Analysis and Design, Class Relations

Prof. Dr.-Ing. Peter Fromm



Content

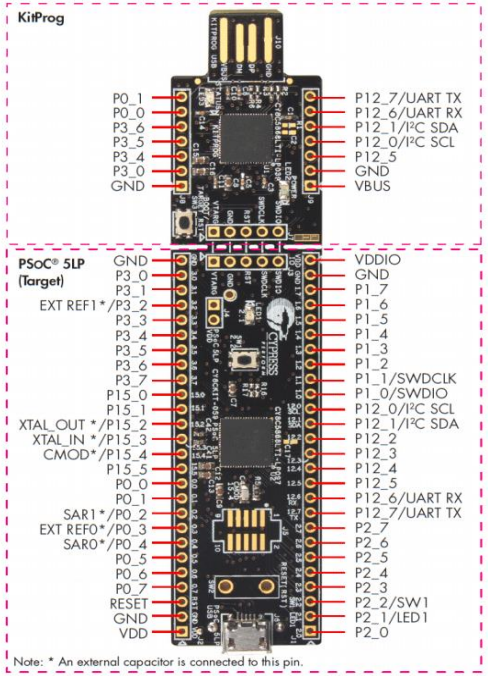
- From structs to classes
- Class Relations
  - Composition
  - Aggregation
  - Association
  - Dependency
- Creating a first OO architecture

Let’s develop a sensor driver for a Lidar sensor

Requirements

- The Lidar transmits data packages via the UART port to the controller.
- The data packages are translated into polar coordinates (angle, distance).
- A 360° scan consists of a variable number of polar coordinates, depending on the rotation speed.
- The following algorithms will use the data
  - Collision detection (in the driving direction)
  - Scan visualisation
  - FullScan transmission

How would we solve this in C?



UART

Let’s develop a sensor driver for a Lidar sensor
Some code snippets

```
typedef struct {
    uint8_t quality;
    uint16_t angle;
    uint16_t distance;
} LIDAR_scanvalue_t;

typedef struct {
    uint16_t readIndex;
    uint16_t writeIndex;
    uint16_t countRead;
    LIDAR_scanvalue_t scan[LIDAR_MAXSAMPLEPERSCAN];
} LIDAR__userScan_t;
```

```
 if (((int32_t)LIDAR__Sensor.scan.data[LIDAR__Sensor.scan.currentWriteArray].scan[writeIndex-1].angle
- (int32_t)scanValue->angle) > 10000)
    {
        LIDAR__Sensor.scan.data[LIDAR__Sensor.scan.currentWriteArray].readIndex = writeIndex;
    }
```

Problem: Data and code is separated. User has to know and understand the detailed data structure in order to access it.

C++ Class

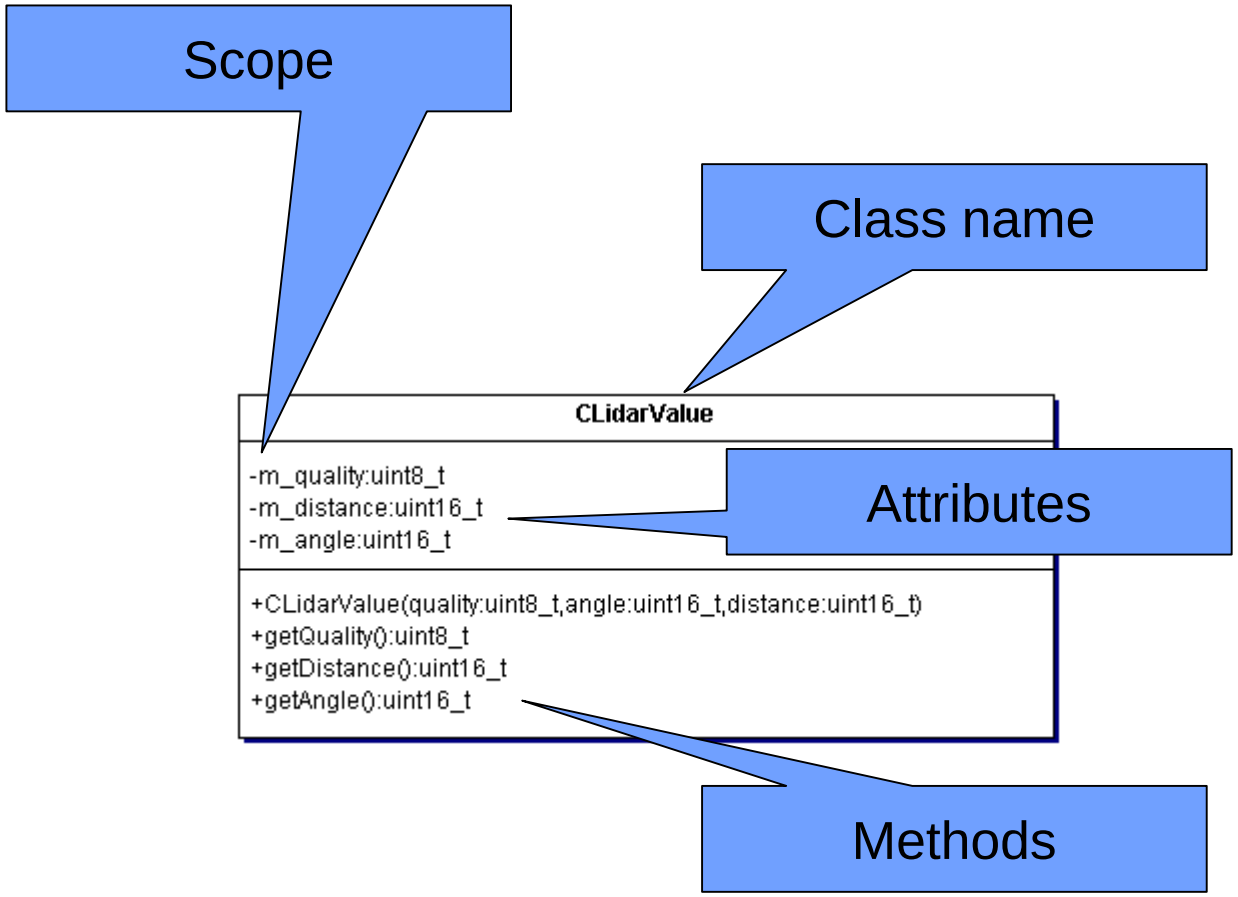
Whereas a C-Struct only contains data, a C++ class contains data and functions (methods) to access the data.

```
class CLidarValue {
private:
    uint8_t m_quality;
    uint16_t m_distance;
    uint16_t m_angle;
public:

    CLidarValue(uint8_t quality, uint16_t angle, uint16_t distance);
    uint8_t getQuality();
    uint16_t getDistance();
    uint16_t getAngle();

};
```

Private attributes

Public methods

Class representation in UML



Naming

- Classes start with a capital C to distinguish them from objects
- Attributes start with a small m (member) to distinguish them from local variables and parameters

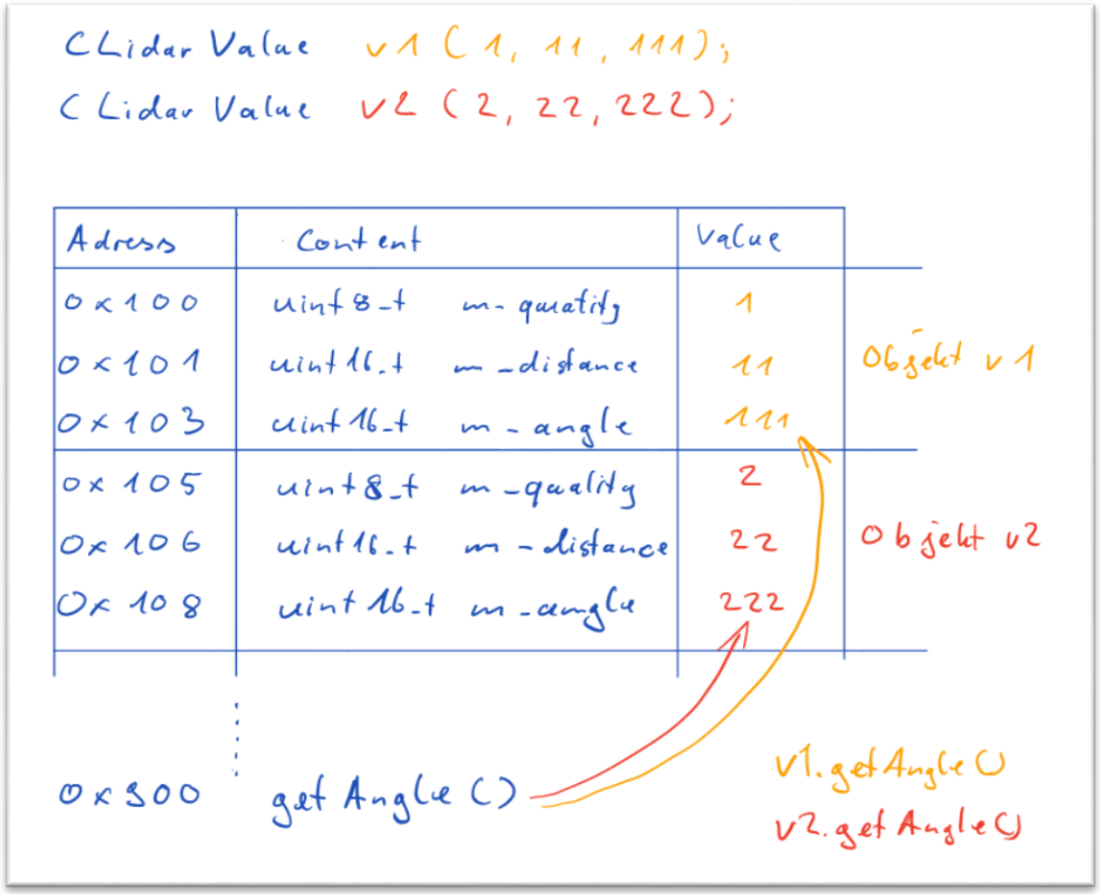
Structure

- Attributes typically are private, i.e. they cannot be changed from the outside directly.
- Access is only provided by public methods, which gives us more options for error handling (e.g. checking for wrong parameters)

Classes and Objects

- Objects are instances of classes.
- When creating an object of a class, the memory as required by the class structure will be reserved for the object.
- A method of a class is always called with object binding, so that the method knows which data it should process (remember the me-Pointer in C?)

Let's create a first class and some objects!



Let’s develop a sensor driver for a Lidar sensor
Object Oriented Analysis

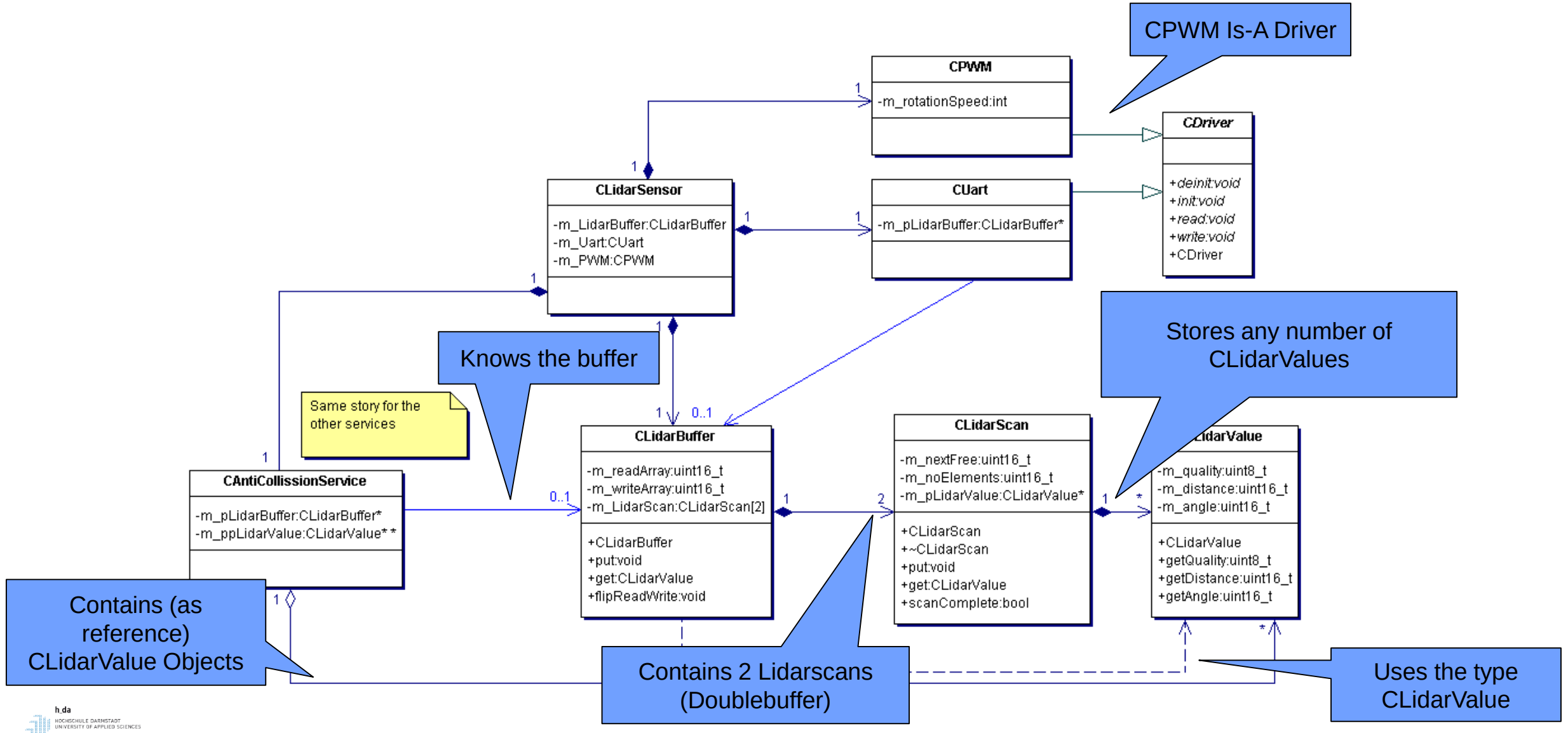
- Identify potential objects or attributes by looking at the nouns.
- Identify possible class relations by looking at terms like “contains”, “knows”, “uses”
- Identify possible verbs describing potential methods.

Let’s create a first high level class diagram!

Requirements

- The Lidar transmits data packages via the UART port to the controller.
- The data packages are translated into polar coordinates (angle, distance).
- A 360° scan consists of a variable number of polar coordinates, depending on the rotation speed.
- The following algorithms will use the data
  - Collision detection (in the driving direction)
  - Scan visualisation
  - FullScan transmission

A first class hierarchy



Complexity is hidden in methods

C Style

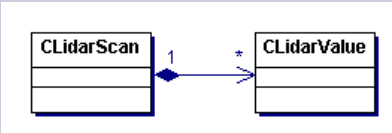
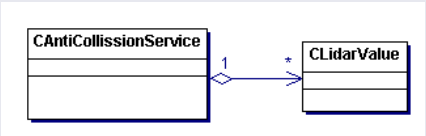
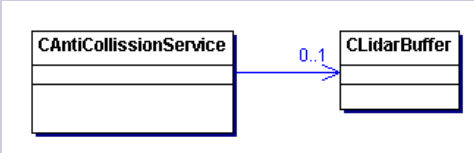
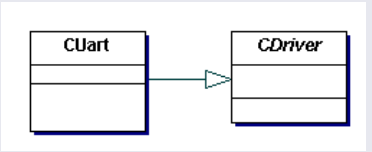
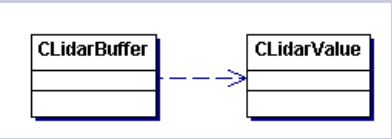
```
if (((int32_t)LIDAR__Sensor.scan.data[LIDAR__Sensor.scan.currentWriteArray].scan[writeIndex-1].angle -
(int32_t)scanValue->angle) > 10000)
    {
        LIDAR__Sensor.scan.data[LIDAR__Sensor.scan.currentWriteArray].readIndex = writeIndex;
    }
```

C++ Style

```
CLidarValue myValue = mySensor.get();
if (myValue.getAngle() – scanValue.getAngle() > 10000)
{
     //Scan completed, flip read and write array
     mySensor.flipReadWrite();
}
```

Note: At this level, we don’t care about the details!

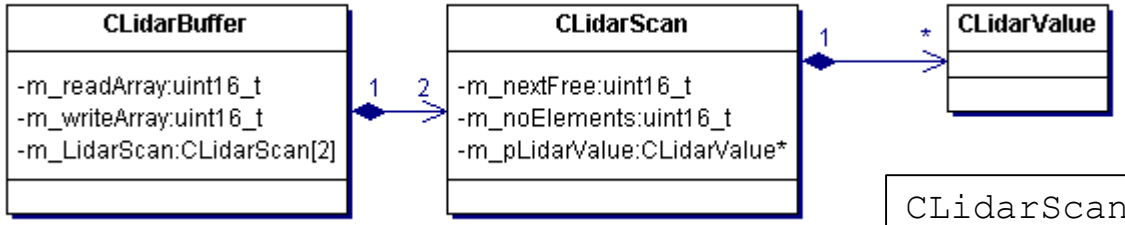
Class relations

Relation	Description	Representations
Composition	Part-whole relation (whole object provides memory for the parts)	
Aggregation	Part-whole relation (parts are stored in another object, whole object contains references)	
Association	One object knows another	
Inheritance	One class is a specialisation of another class	
Dependency	One class uses the type of the other class (e.g. as local variable or parameter)	

Composition

A composition is a **strong** part-whole relation.

- The part object only exists as long as the whole object exists
- In technical terms: the whole object provides the memory for the part, either by declaration or allocation (new)




```
class CLidarBuffer {
private:
    uint16_t m_readArray;
    uint16_t m_writeArray;
    CLidarScan m_LidarScan[2];
public:
    CLidarBuffer();
    void put(CLidarValue const& data);
    CLidarValue get();
    void flipReadWrite();
};
```

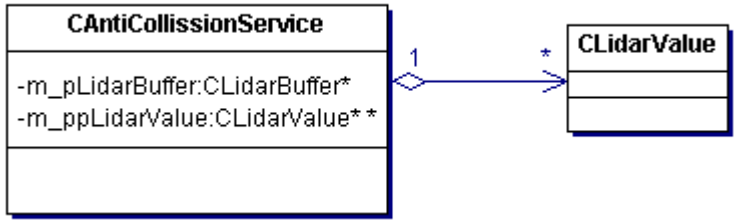
```
class CLidarScan {
private:
    uint16_t m_nextFree;
    uint16_t m_noElements;
    CLidarValue* m_pLidarValue;
public:
    CLidarScan(uint16_t noElements);
    ~CLidarScan();
    void put(CLidarValue data);
    CLidarValue get();
    bool scanComplete();
};
```

```
CLidarScan::CLidarScan(unsigned int
noElements){
m_pLidarValue = new CLidarValue[noElements];
}
```

Aggregation

An Aggregation is a **weak** part-whole relation.

- The part object exists independently of the whole object.
- In technical terms: the whole object stores references or pointers to the child objects



We store pointers. The objects themselves are stored in a different class

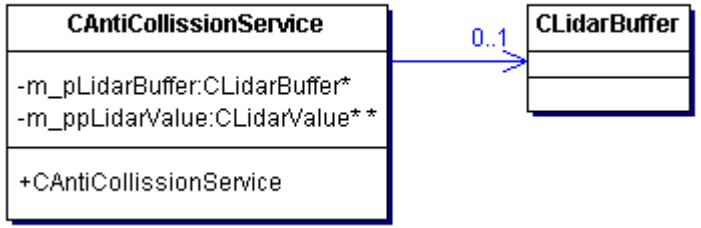
```
class CAntiCollissionService {
private:
    CLidarBuffer * m_pLidarBuffer;
    CLidarValue ** m_ppLidarValue;
};
```

```
CAntiCollissionService::CAntiCollissionService(uint16_t
maxLidarValues)
{
m_ppLidarValue = new CLidarValue*[maxLidarValues];
}
```

Association

An Aggregation is “know” relation between to objects.

- The objects exist independently of each other and have a reference to the other.
- In technical terms: the object stores a reference or pointer to the other object.

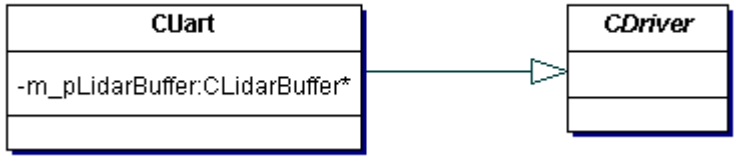



```
class CAntiCollissionService {
private:
    CLidarBuffer * m_pLidarBuffer;
    CLidarValue ** m_ppLidarValue;
};
```

Inheritance

An Inheritance allows specialisation and generalisation of classes and supports reuse.

- The specialised object has all capabilities of the derived object but may add or change some capabilities.




```
class CUart : public CDriver {
private:
    CLidarBuffer * m_pLidarBuffer;
};
```

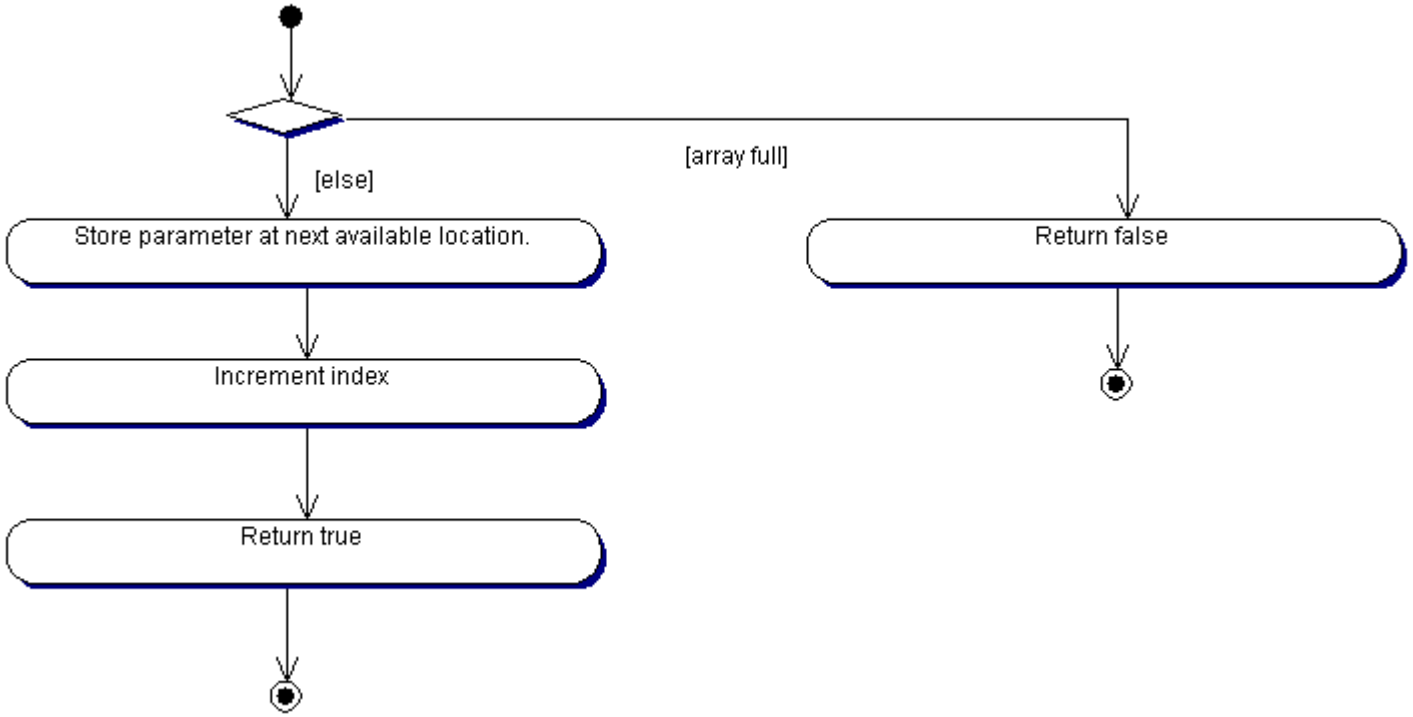
Some rules

- Child objects realised by declaration always represent compositions.
- A pointer member can indicate (almost) any relation: composition, aggregation or association
- Association and Aggregation share the same implementation, the only differ in terms of the design.
- Dependencies are the weakest form of relations. They are only visualized in case no other relation does exist between the classes.
- Every object needs to be created somewhere, i.e. there should be 1 (but typically only 1) composition relation for every class. Either direct or indirect through inheritance.

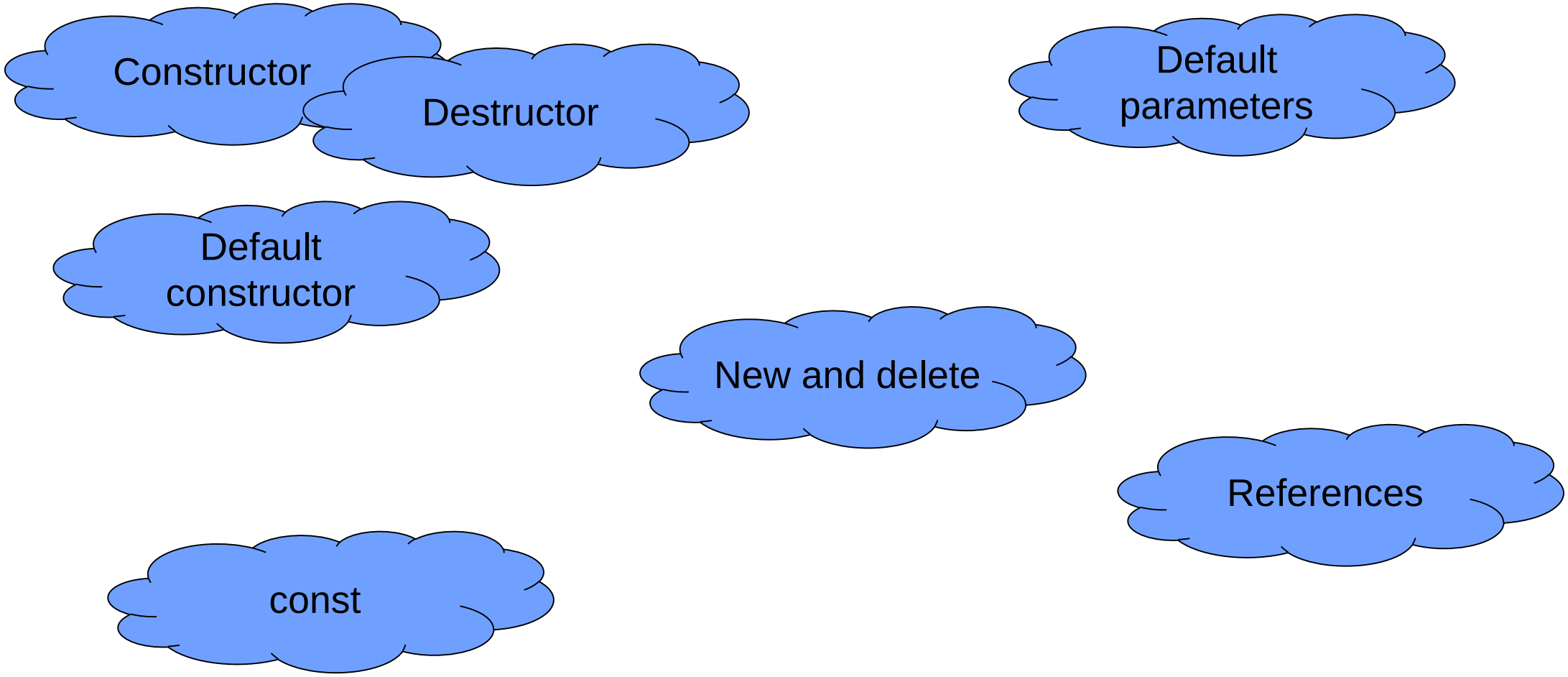
A first algorithm…

The CLidarScan::put method shall fulfil the following requirements:

- The parameter value shall be stored at the next free location
- In case the storage was successful, true shall be returned
- In case no memory is available, false shall be returned



Some additional stuff we came across…



Wrap-Up

- Concept of object oriented analysis
- Classes
- Class relations
- Activity diagram
- Some C++ specific stuff

This lecture is pretty fundamental for the rest of the semester. It is recommended to program the example at home from scratch and check the relevant concepts either in a textbook or at www.cplusplus.com .